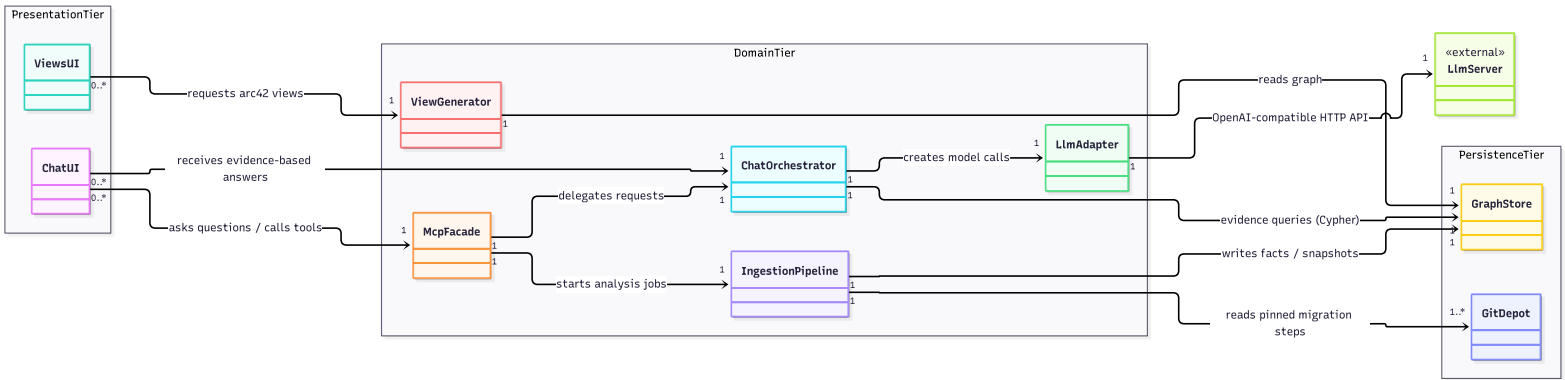
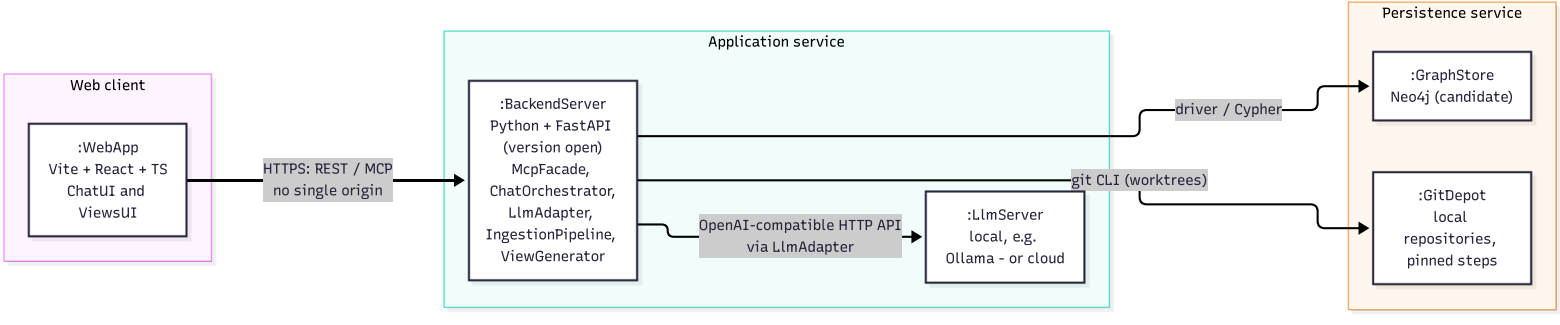
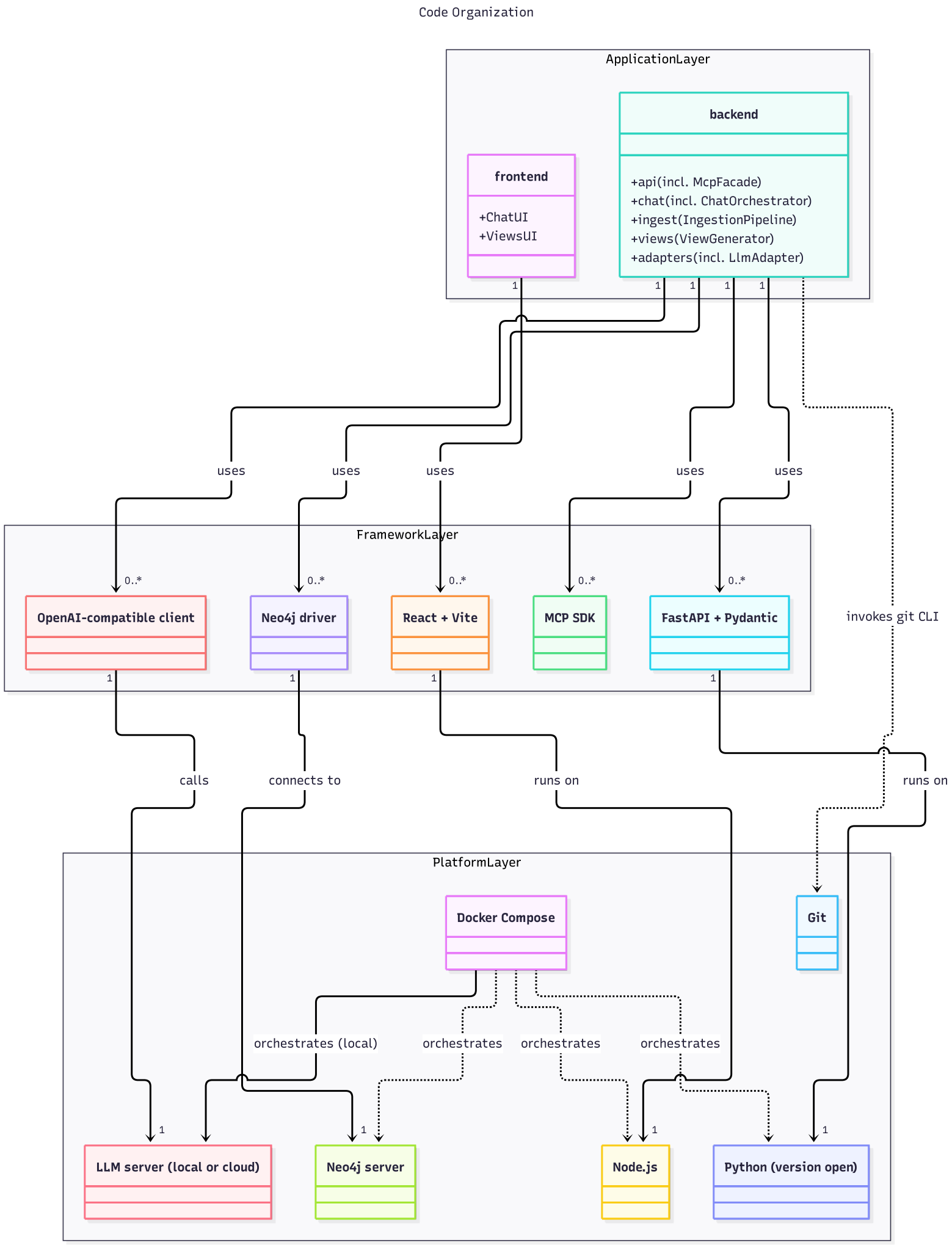

Runtime Architecture
PresentationTier
ViewsUI
ChatUI
requests arc42 views
receives evidence-based answers
asks questions / calls tools
DomainTier
ViewGenerator
ChatOrchestrator
McpFacade
IngestionPipeline
delegates requests
starts analysis jobs
creates model calls
LlmAdapter
reads graph
OpenAI-compatible HTTP API
«external» LlmServer
evidence queries (Cypher)
writes facts / snapshots
reads pinned migration steps
PersistenceTier
GraphStore
GitDepot

Objects at Runtime




Code Organization
ApplicationLayer
backend
+api(incl. McpFacade)
+chat(incl. ChatOrchestrator)
+ingest(IngestionPipeline)
+views(ViewGenerator)
+adapters(incl. LlmAdapter)
frontend
+ChatUI
+ViewsUI
uses
uses
uses
uses
uses
invokes git CLI
FrameworkLayer
OpenAI-compatible client
Neo4j driver
React + Vite
MCP SDK
FastAPI + Pydantic
calls
connects to
runs on
runs on
PlatformLayer
Docker Compose
Git
orchestrates (local)
orchestrates
orchestrates
orchestrates
LLM server (local or cloud)
Neo4j server
Node.js
Python (version open)

Summary of the Underlying Technology Stack

SunsetTracker is a locally reproducible prototype: every component below runs on a single machine (Docker Compose or plain processes) and is licence- compatible with MIT. The stack follows the SBOM Vision A (permissive) decision record; versions marked *unpinned* are resolved at the first lockfile.

Area	Technology	Role in the system	Status
Frontend	Vite + React + TypeScript	Web app with the AI chat interface (ChatUI) and the arc42 architecture views (ViewsUI); built and hosted separately from the backend (no single origin)	fixed
Backend	Python 3.12 + FastAPI + Pydantic	Application server: MCP tool facade, chat orchestration, ingestion pipeline, arc42 view generation; Pydantic validates the canonical fact schema	fixed (Python 3.12 per SBOM Vision A)
Chat interface	MCP SDK (`mcp`)	Tool protocol between the chat host and the backend tools (analyze, compare, history, views)	fixed
LLM access	OpenAI-compatible adapter	Provider-agnostic HTTP boundary; encapsulates the model call and makes local and cloud runtimes interchangeable	fixed (adapter), runtime open
LLM runtime	Ollama 0.40.9 (local) or a cloud provider via API key	Inference for chat answers and RAG; local runtime preferred for the offline demo, cloud possible later without code changes thanks to the adapter	local for MVP, cloud optional
Chat model	gpt-oss-20b (revision unpinned)	Open-weight chat model served by the local runtime	fixed (model), revision open
Embeddings	Qwen3-Embedding-0.6B (revision unpinned)	Optional local embeddings for retrieval-augmented answers	optional
Knowledge store	PostgreSQL 17 + Apache AGE 1.7.0 + pgvector	Graph store for code, domain and process facts (openCypher via `cypher()`) plus vector index for document chunks; single database for facts, jobs and manifests	fixed (Vision A)
Code sources	Git (local repositories / self-contained bundles)	Read-only input for the ingestion pipeline; migration steps are pinned by full commit SHA; no builds or execution of analyzed code	fixed
Code analysis	code-review-graph 2.3.9, SQLGlot 30.22.0, markdown-it-py	Deterministic extractors for code structure, SQL access and documentation; no LLM involved in ingestion	fixed
Chat persistence	LibreChat 0.8.8 + FerretDB 2.x + PostgreSQL 17 (DocumentDB)	Chat host UI and its document store; FerretDB replaces MongoDB (SSPL) to keep the stack permissively licensed	fixed
Testing	pytest	Unit, contract and integration tests with golden-file view tests	dev-only
Supply chain	cyclonedx-bom, Syft, pip-licenses	SBOM generation and licence gate in CI	dev-only
Orchestration	Docker Compose	Local deployment of all services; one-command reproducibility for development and demo day	fixed

Explanation of the Diagrams and Architectural Choices

Runtime Architecture (page 1)

The runtime architecture is organized in three tiers. The **presentation tier** contains the two frontend features: the chat interface (ChatUI) and the arc42 architecture views (ViewsUI). The **domain tier** holds the backend logic: the MCP tool facade (McpFacade) exposes the backend capabilities as tools, the chat orchestrator (ChatOrchestrator) combines retrieval with model calls, the ingestion pipeline (IngestionPipeline) analyses pinned migration steps, and the view generator (ViewGenerator) projects the knowledge graph into arc42 views. The **persistence tier** contains the knowledge graph (GraphStore) and the local Git depot (GitDepot) with the pinned repository states.

Two boundaries are deliberate. First, the frontend and the backend are built and hosted **separately** (no single origin), because the frontend offers more than the chat interface — the arc42 views are a first-class feature. Second, the **LLM server is external**: it may run locally (Ollama) or in the cloud via an API key. The backend talks to it only through the provider-agnostic, OpenAI-compatible LlmAdapter, which is a module of the backend — not a separate runtime component. This keeps the model runtime exchangeable without touching the orchestration logic.

Objects at Runtime (page 2)

The second diagram shows one concrete instance per tier, following the lecture template. The web client runs one `:WebApp` instance (Vite + React + TS, containing ChatUI and ViewsUI). The application service column contains two separate servers: the `:BackendServer` (Python + FastAPI, hosting McpFacade, ChatOrchestrator, LlmAdapter, IngestionPipeline and ViewGenerator as modules) and the `:LlmServer` (local, e.g. Ollama — or a cloud service). The persistence service column contains the `:GraphStore` (PostgreSQL 17 with Apache AGE) and the `:GitDepot` with the local repositories. The LLM server is grouped under "application service" because it provides inference, but it remains a separate server from the backend.

Code Organization (page 3)

The code organization maps the same components onto three horizontal layers. The **application layer** contains our own code in two separate repositories: `frontend/` (ChatUI, ViewsUI) and `backend/` (api, chat, ingest, views, adapters). The **framework layer** contains the libraries we use and configure: React + Vite, FastAPI + Pydantic, the MCP SDK, the PostgreSQL/AGE driver and the OpenAI-compatible client. The **platform layer** contains the system software we only operate: Node.js, Python 3.12, the PostgreSQL server, Git, the LLM runtime and Docker Compose. The LLM runtime sits in the platform layer because we run or consume it, but never modify its code.

Key choices and their rationale

Choice	Rationale
Separate frontend and backend hosting	Industry partner requirement; the frontend is more than a chat shell (arc42 views). No single origin.
MCP as the chat-to-backend interface	A typed tool boundary keeps the model from inventing backend behavior; the chat host executes tool calls, the backend owns the logic.
OpenAI-compatible LLM adapter	Encapsulation and exchangeability: local Ollama for the offline demo, a cloud provider via API key later — no changes to orchestration code.
PostgreSQL 17 + Apache AGE as knowledge store	One permissively licensed database serves graph queries (openCypher), vector search (pgvector) and job/manifest storage; matches SBOM Vision A.
Deterministic ingestion without LLM	Facts (code structure, SQL access, docs) are extracted deterministically and validated with Pydantic; the LLM only explains, it does not invent facts. Evidence-based answers are the core requirement.
Pinned migration steps (full commit SHAs)	Every question about migration status is answered against a reproducible snapshot; Git bundles make the reference system shareable and offline-capable.
Local-first, MIT-compatible stack	Demo day must work offline (Ollama); every dependency is permissively licensed (FerretDB replaces MongoDB/SSPL).